

Codex 바이트 코딩

Q&A + 유의사항

처음 시작하는 학습자를 위한 짧은 운영 기준

SECTION 01

시작 전 기준

모델, 대안 도구, 첫 진입 경로

MODEL

**처음에는 GPT-5.4 + medium을 기본으로 쓰고,
필요할 때만 올린다.**

처음부터 최고 설정으로 시작하기보다,
문제 난이도에 따라 단계적으로 올리는 편이 효율적이다.

Start simple, then raise the model only when the work justifies it.

Codex가 막힐 때의 기본 대안

먼저 할 일

- Codex Desktop이 막히면 VS Code 계열 IDE의 Codex IDE extension으로 이어서 작업한다.
- 가능하면 같은 폴더를 열고 같은 맥락으로 이어간다.
- 핵심은 작업 흐름을 끊지 않는 것이다.

기억할 점

- 도구가 바뀌어도 프로젝트 맥락은 유지해야 한다.
- 새로 시작하지 말고, 이어서 진단하게 만든다.
- 폴더와 목적을 먼저 읽히는 것이 중요하다.

Switch the surface if needed, but keep the same workspace context.

ALTERNATIVE

**Antigravity 같은 대안도 가능하지만,
무료 범위와 정책은 바뀔 수 있다.**

Codex를 바로 쓰기 어렵거나 다른 IDE 흐름이 필요하면 대안 도구를 쓸 수 있다.
다만 미리보기 정책, 무료 사용 범위, 제공 모델은 최신 안내를 함께 확인하는 것이
좋다.

Policy and model availability can change.

SECTION 02

실전 운영 팁

계획, 감수, 인수인계, 문제 해결

초반에 꼭 쓰는 세 가지

01

Plan Mode

사소한 부분까지 설명하고, 빠르게 구현으로 들어가자 말고, 모르는 부분은 질문으로 되돌려 달라고 한다.

02

AGENTS.md

목표, 범위, 수정 규칙, 금지 작업, 완료 기준을 먼저 적어 프로젝트 헌법으로 쓴다.

03

중간 감수

비전공자는 결과를 빨리 보지만 구조적 오류를 놓치기 쉬우므로 중간마다 시니어 관점 감수를 받는다.

Early quality comes more from structure than from typing speed.

길어지거나 막힐 때의 대응

01

남은 단계 확인

남은 단계를 숫자로 나누고, 최종 확인 전까지는 묻지 말고 계속 진행하게 한다.

02

hand-off.md

현재 목표, 끝난 것, 남은 것, 다음 첫 단계를 문서로 남긴다.

03

근원 원인 해결

증상만 덮지 말고 5 Whys, 시니어 관점, QA 관점으로 원인까지 내려가 해결한다.

When the thread gets long, reorganize the work instead of pushing harder.

공유와 로컬 AI 사용 기준

공유

- 단일 HTML이면 파일 하나로 전달한다.
- 여러 파일이 함께 있으면 폴더째 압축해 전달한다.
- 연습용 공개 배포는 GitHub Pages가 가장 쉽다.

보안과 로컬 AI

- 민감한 코드나 데이터는 로컬 모델 사용을 검토할 수 있다.
- Ollama의 gemma4:e4b와 Continue 조합으로 로컬 실습이 가능하다.
- Ollama: 모델호스팅 (로컬서버)
- VS Code: 편집기 (개발환경)
- Continue: 인터페이스 (AI연결)

Easy sharing is useful, but security-sensitive services need stronger deployment rules.

CLOSING

**처음부터 완벽하게 쓰는 사람보다,
덜 무너지게 운영하는 사람이 더 빨리 간다.**

- 처음에는 Codex Desktop으로 익숙해진다.
- 필요할 때만 모델, 도구, 로컬 환경을 넓힌다.
- 중요한 것은 속도보다 맥락 유지와 구조화다.

Keep the context, then expand the tools.